



SH320 Data Engineering

Spring 2026 Semester

9 Outlier Detection

Hyun Ahn

School of Computing and Artificial Intelligence

hyunahn@hs.ac.kr

Core Libraries for Data Science



- **[W2] N-Dimensional Arrays**
 - NumPy arrays
 - Array manipulation
- **[W3] Efficient Computation in NumPy**
 - Universal functions
 - Advanced features

- **[W5] Data Manipulation: Part 1**
 - Pandas objects: Series, DataFrame
 - Implicit / explicit indexing
- **[W6] Data Manipulation: Part 2**
 - Pandas objects & manipulation
 - Combining datasets
 - Exploratory data analysis

Data Engineering

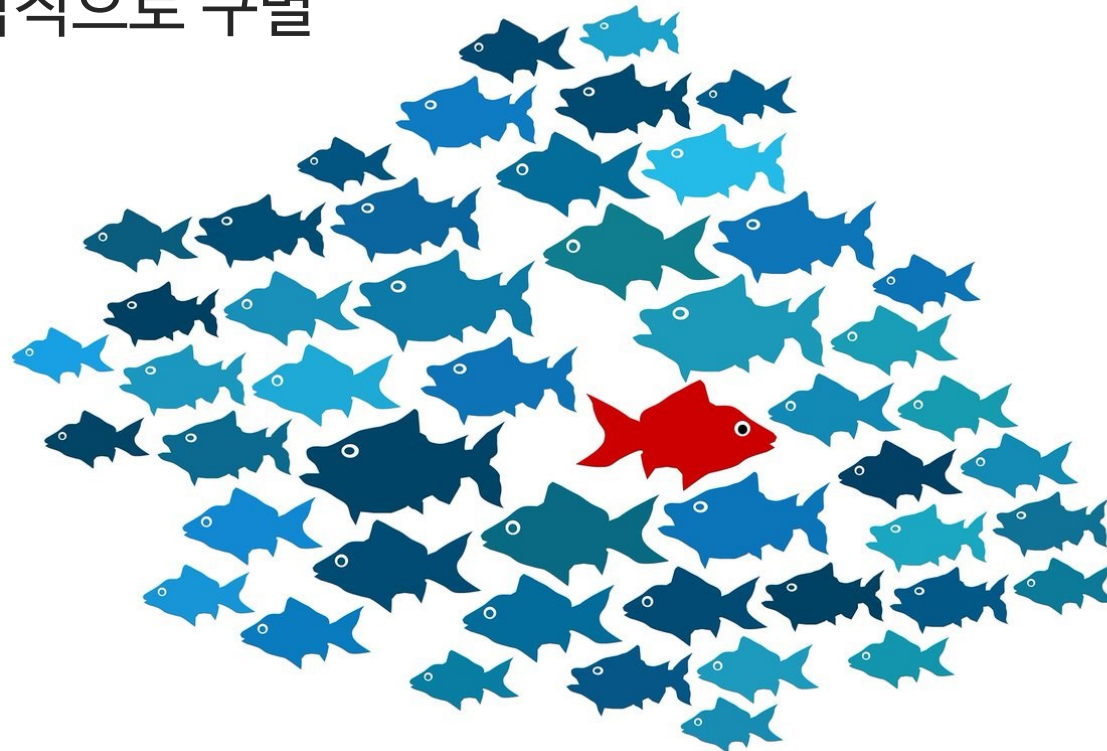


- **[W7] Data Acquisition**
 - Web scraping
 - RESTful API
- **[W9] Working with Time Series**
 - Time series manipulation
 - Time-series-specific operations
- **[W10,11] Handling Missing Data**
 - Missing data problems
 - Processing missing data in Python
- **[W12] Outlier Detection**
 - Outlier detection problem
 - Processing outliers in Python

9.1 이상치 탐지 문제

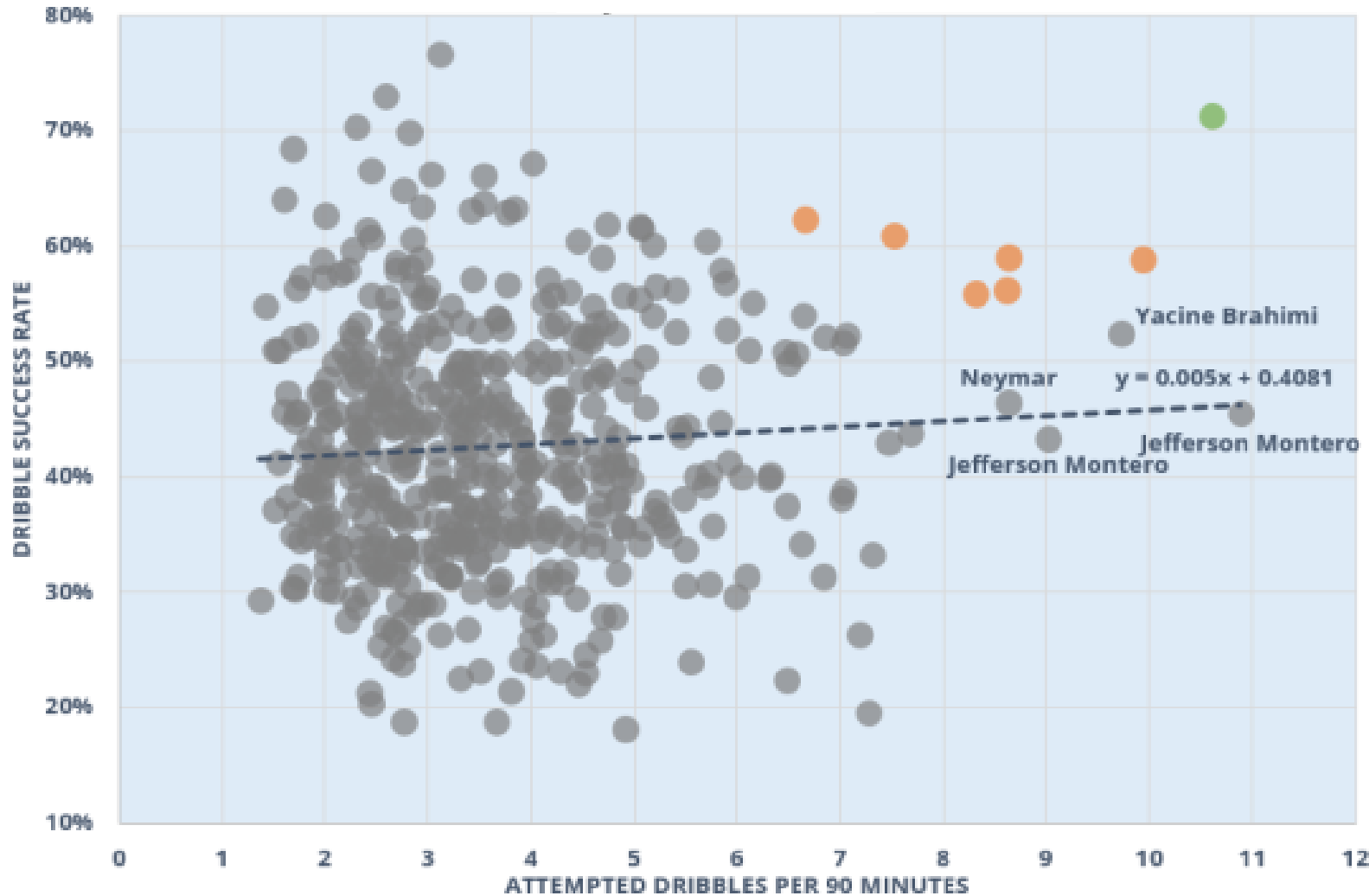
What Are Outliers?

- **이상치** Outliers : 통계적으로 **다른 데이터들과 큰 편차를 보이는 관측값**
 - 데이터의 전체 패턴에서 벗어난 관측값이므로, 일반적으로 박스 플롯 또는 산포도를 통해 시각적으로 구별



Source: towards data science

ex) The Greatest Outlier in Soccer



스페인라리가 09/10–14/15 시즌
선수 492명의 드리블 시도 vs 성공율

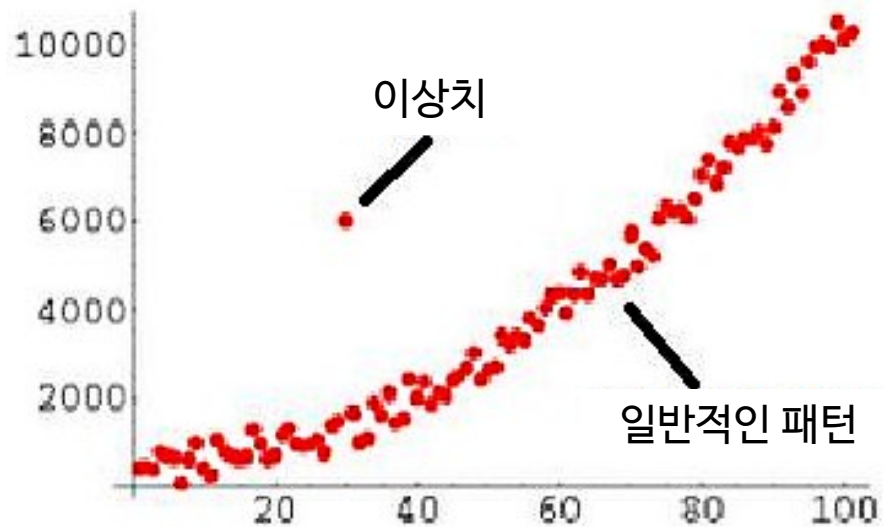
orange: 메시 09/10 – 14/15 시즌

green: 메시 15/16 시즌

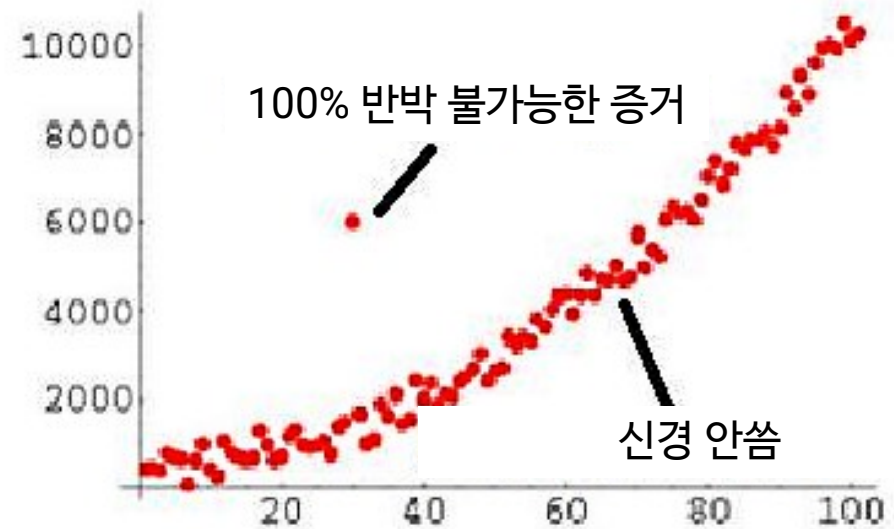
Source: barcanumbers

ex) Two Different Views of Outliers

내가 보는 관점



안티백서/GMO반대론자/유사과학자들이 보는 관점



Why Do They Occur?



이상치 발생 유형	원인	예
데이터 입력 오류 (data entry errors)	사람에 의한 실수 (human errors)	천원 단위 숫자를 원 단위로 입력
측정 오류 (measurement errors)	측정장치의 비정상 작동 (instrument errors)	온도센서 고장
실험 오류 (experimental errors)	실험 설계/수행 과정에서의 실수	운동선수에 대한 피지컬 테스트에서 동일하지 않은 실험조건(기후, 장소) 설정
의도적인 이상치 (intentional outliers)	실험 목적 또는 불편한 설문 응답	이상치 탐지 모델 테스트를 위해 고의로 이상치들을 생성하여 실험
데이터 처리 오류 (data processing errors)	데이터 조작 실수 또는 의도하지 않은 데이터 변형	원달러 환율(예: 1,180.95) 데이터 크롤링에서 달러원 환율(예: 0.000847) 데이터를 저장
표본추출 오류 (sampling errors)	잘못된/편향된 표본 추출	표본 크기, 집단을 고려하지 않은 여론조사
자연적 이상치 (natural outliers)	자연적으로 발생	와우 시그널, 지진파 신호

What Do They Affect?



- 전체 데이터의 통계 특성 왜곡 $\Rightarrow$ 기초 통계 분석 결과의 신뢰도 저하(예: 평균)
- 기초 통계에 기반한 다른 고급 통계 분석 결과의 신뢰도 저하
- 데이터 기반 모델의 성능(예: 예측/분류 정확도) 저하
- 잘못된 의사결정 유도

Applications



- 이상치 탐지 및 제거의 기본 목적: **데이터 품질의 향상**
- 관련 응용:
 - **이상거래 탐지** Fraud Detection
 - **사이버보안 침입 탐지** Intrusion Detection in Cyber Security
 - **의료 데이터 분석** Medical Data Analysis
 - **환경·재해 모니터링** Environment Monitoring

이상치 탐지 문제 기준

- 데이터의 차원
 - 단변량Univariate, 이변량Bivariate, 다변량Multivariate
- 분포 가정 여부
 - **모수적**Parametric 방법: 데이터의 분포에 대한 가정을 포함(예: 정규 분포)
 - 실제 관측값에서 계산된 통계량에 기반(평균, 표준편차)
 - **비모수적**Non-parametric 방법: 데이터의 분포에 대한 가정이 필요하지 않음
 - 실제 관측값보다는 순위에 기반(중앙값, 사분위수)

How to Detect Outliers?



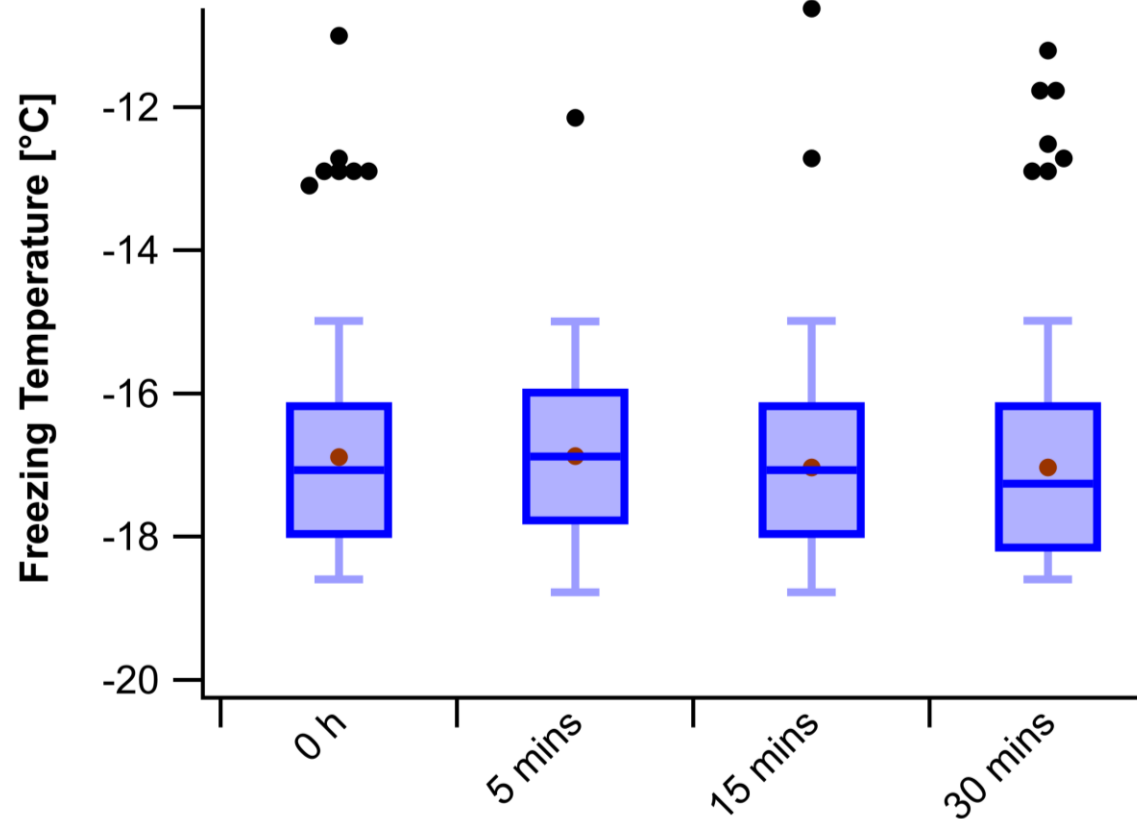
- **이상치 탐지 기법**

- 시각화를 통한 탐지: 박스 플롯^{Box Plot}, 산포도^{Scatter Plot}
- 사분위수 범위 기반 탐지
- **Z-Score** 기반 탐지
- **DBSCAN**
- **Isolation Forest** 등

Visualization Techniques

Box Plot

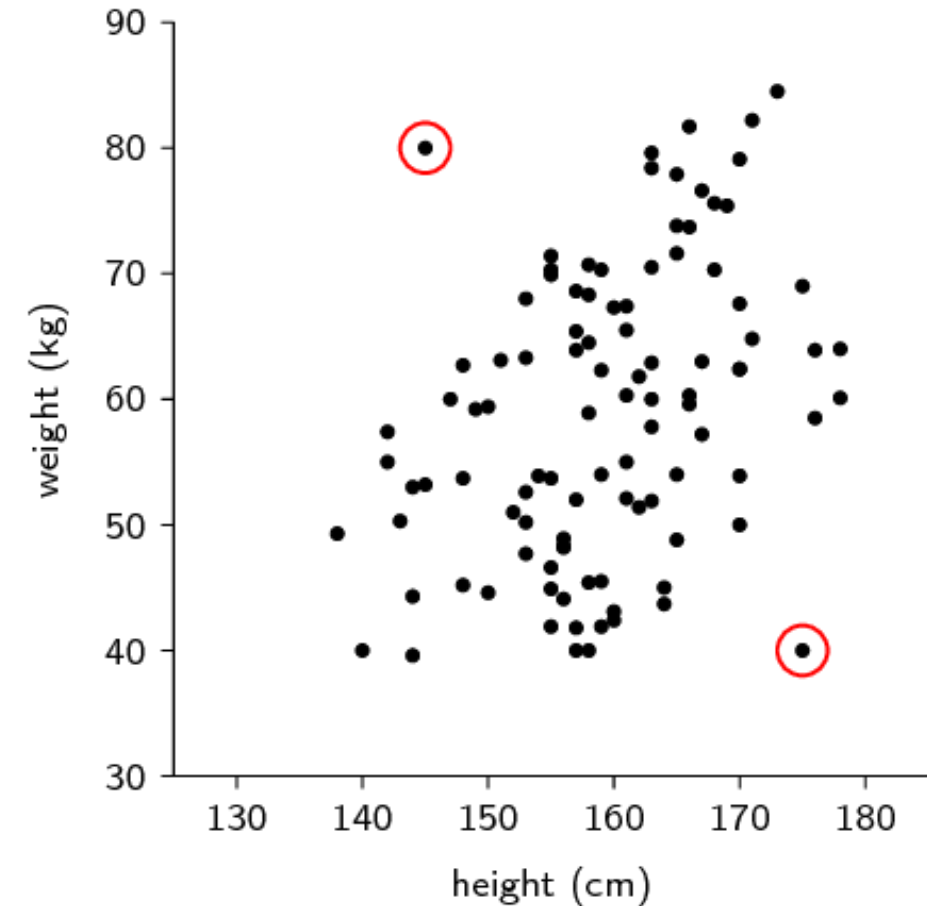
1차원 데이터에 적합



Source: WaveMetrics

Scatter Plot

2, 3차원 데이터 적합



Source: Siyavula

IQR-based Detection

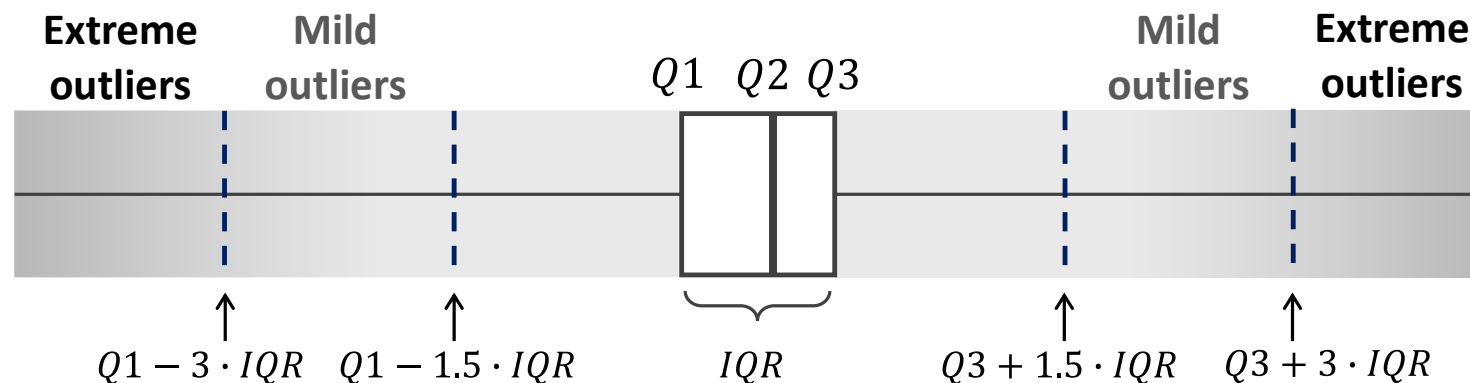
- **사분위수 범위** IQR: Interquartile Range 기반 탐지

- 단변량 데이터에 쉽게 적용할 수 있는 비모수적 방법

- 사분위수: 데이터를 백분위 기준으로 4 등분한 것

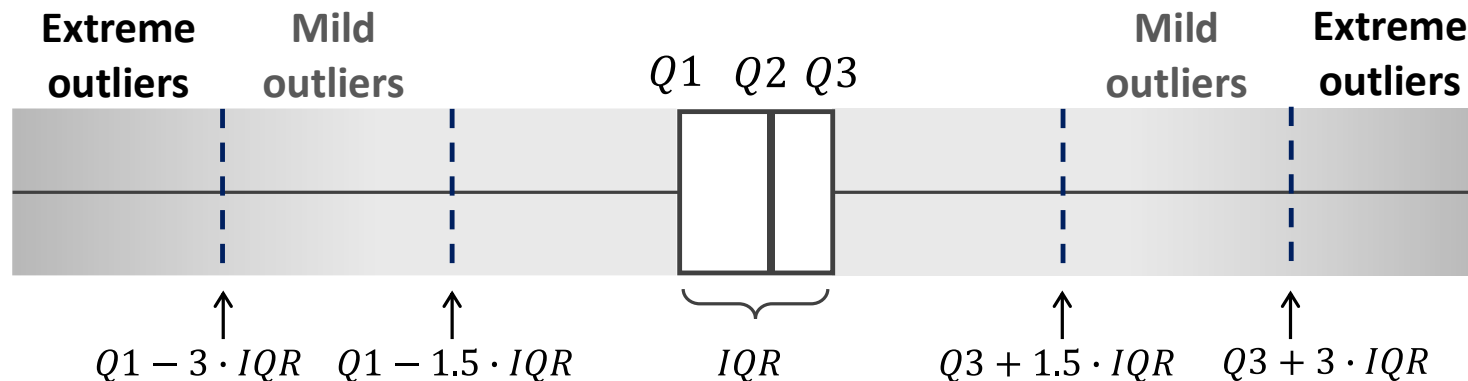
- $Q1$: 하위 25% 값 $Q2$: 하위 50% 값(중앙 값) $Q3$: 하위 75% 값

- 사분위수 범위: $IQR = Q3 - Q1$



IQR-based Detection

- 상한: upper fence = $Q3 + 1.5IQR$
- 하한: lower fence = $Q1 - 1.5IQR$
- 최대값: 상한 이하의 관측 값 중에 가장 큰 값
- 최소값: 하한 이상의 관측 값 중에 가장 작은 값
- 이상치: 상한 보다 큰 값 또는 하한 보다 작은 값



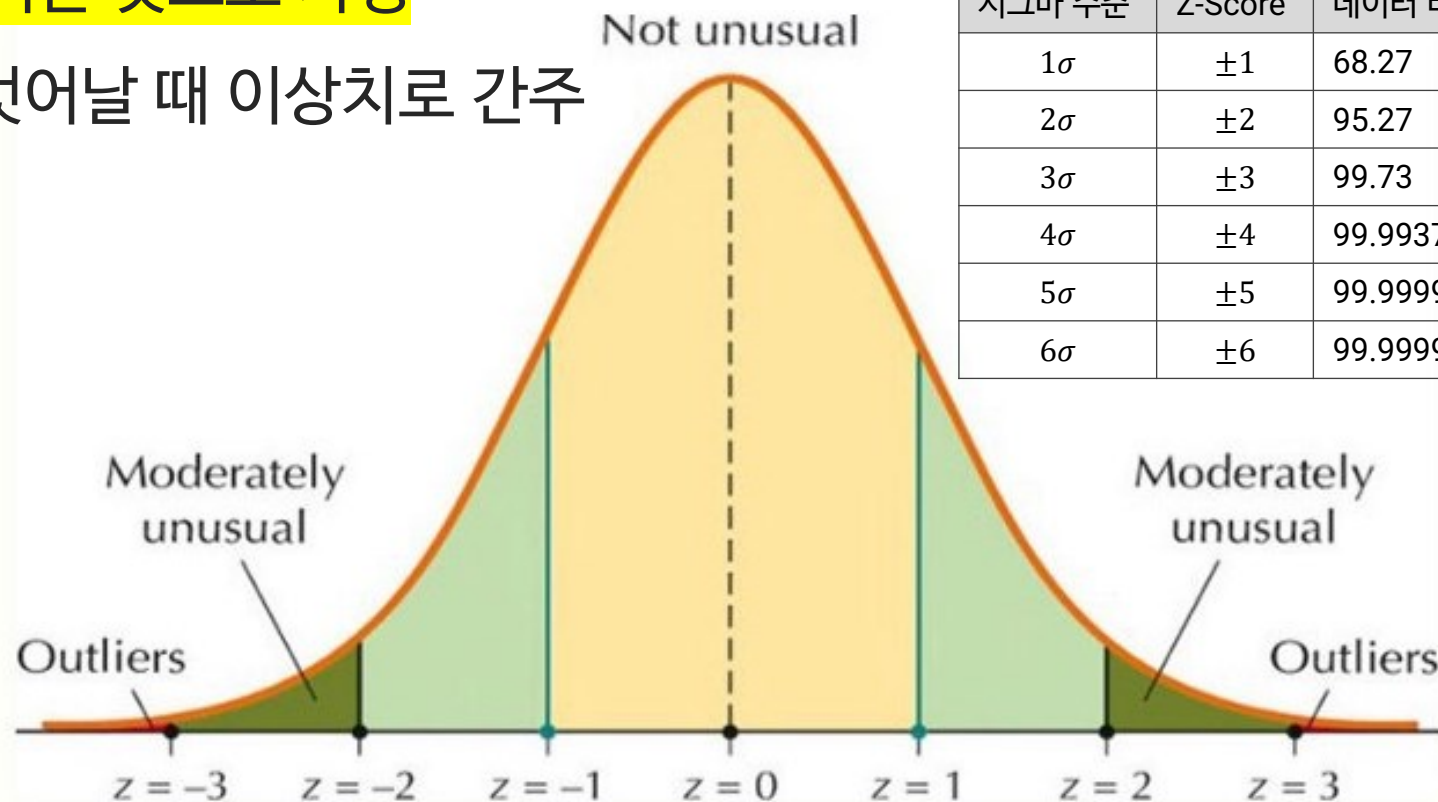
Detecting with Z-Score

• Z-Score

- 단변량 또는 낮은 차원의 데이터에 적합한 모수적 방법
- 데이터가 정규 분포를 가지는 것으로 가정
- 임계치 Z 값(예: ± 3)을 벗어날 때 이상치로 간주

$$Z = \frac{x - \mu}{\sigma}$$

관측값 x 평균 μ
표준편차 σ



시그마 수준	Z-Score	데이터 비율(%)
1σ	± 1	68.27
2σ	± 2	95.27
3σ	± 3	99.73
4σ	± 4	99.9937
5σ	± 5	99.999943
6σ	± 6	99.9999998

Detecting with Z-Score



- Z-Score 예제

100m 달리기 기록

[14, 18, 17, 16, 17, 16, 15, 17, 18, 9]

$N = 10$

$\mu = 15.7$

[2.89, 5.29, 1.69, 0.09, 1.69, 0.09, 0.49, 1.69, 5.29, 44.89]

$\sigma^2 = 6.41$

$\sigma = 2.53$

$Z(x_1) = -0.67$

$Z(x_{10}) = -2.65$

$$\sigma = \sqrt{\frac{\sum (x_i - \mu)^2}{N}}$$

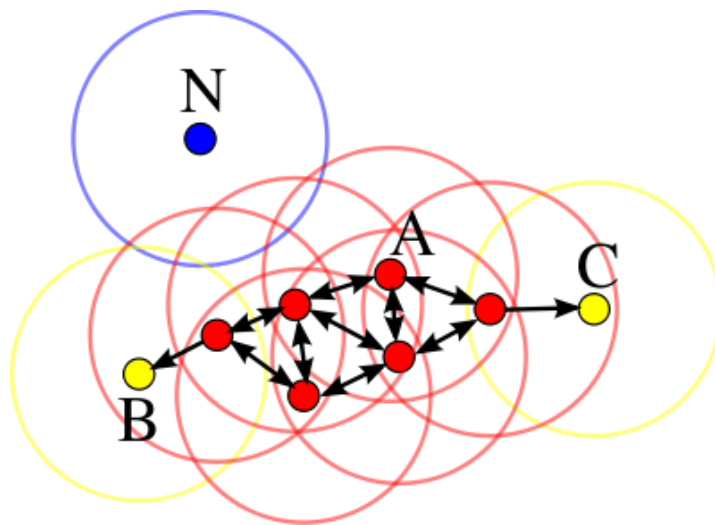
$$Z = \frac{x_i - \mu}{\sigma}$$

Z 임계치: ± 2.5

다변량 데이터에 적합한 이상치 탐지 방법 : DBSCAN, Isolation Forest

• DBSCAN 알고리즘

- 밀도 기반 군집화를 통한 이상치 탐지 기법
 - Epsilon(ϵ) : 군집을 구성하기 위한 최소 거리
 - Min Points : 군집을 구성하기 위한 최소 데이터 포인트 수
- 모든 데이터는 **코어 포인트**, **경계 포인트** 또는 **노이즈 포인트**로 분류됨



Other Advanced Techniques

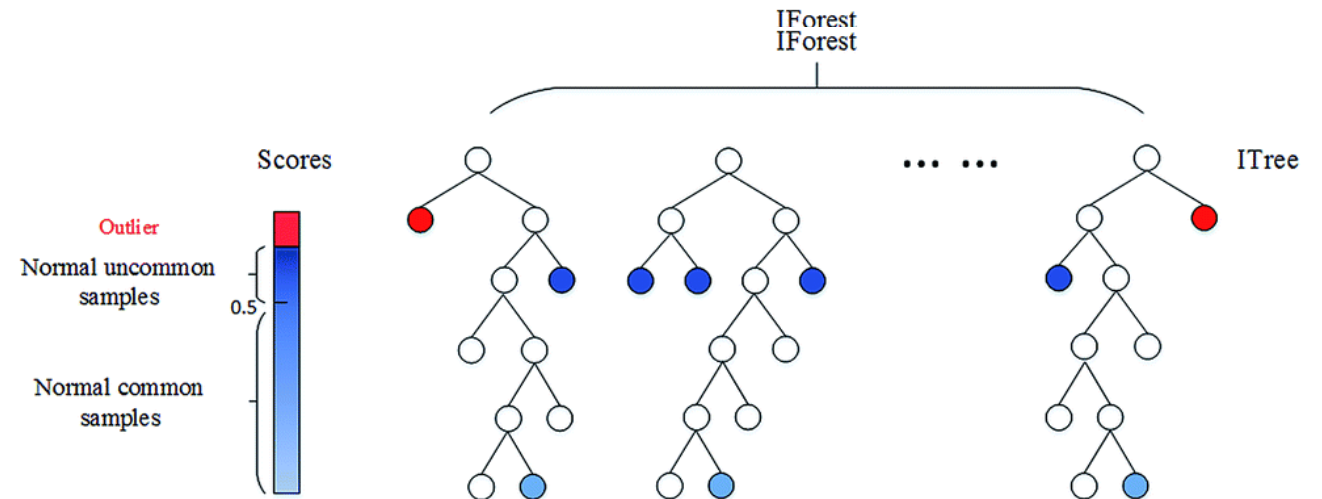
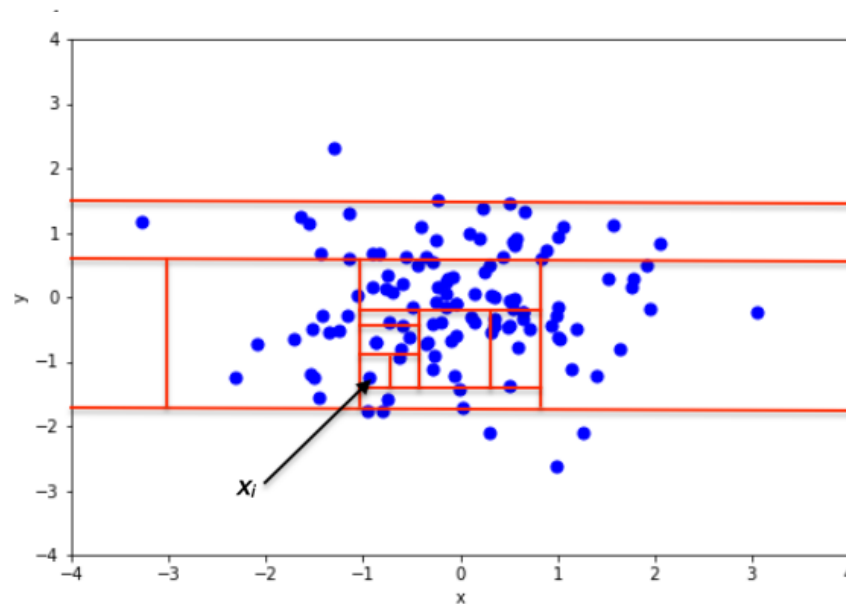
- DBSCAN 알고리즘



Other Advanced Techniques

- Isolation Forest

- 이진 트리 기반 이상치 탐지 기법
- 이진 분할을 통해 데이터 영역을 나누는 개념을 기반으로 함(깊이: 정상데이터 > 이상치)
 - 가정: 이상치에 가까울수록 분할 되기 쉬우며, 정상 데이터일수록 분할 되기 어렵다. (참고: [DEL](#))

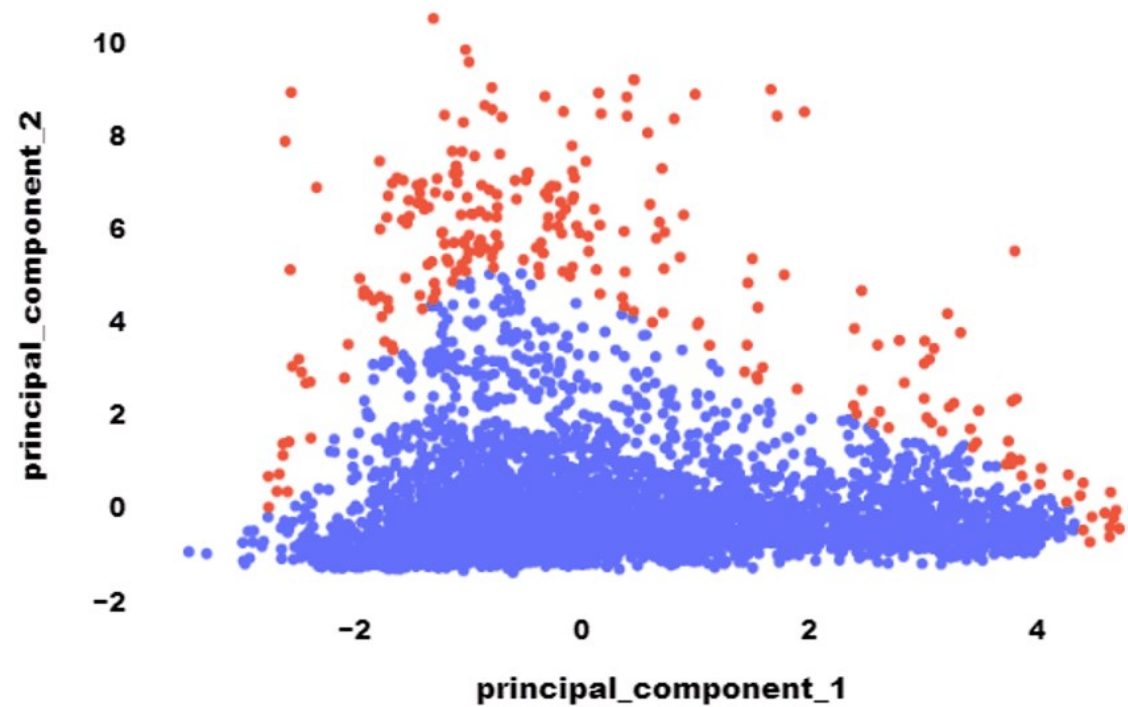


Other Advanced Techniques



- Isolation Forest

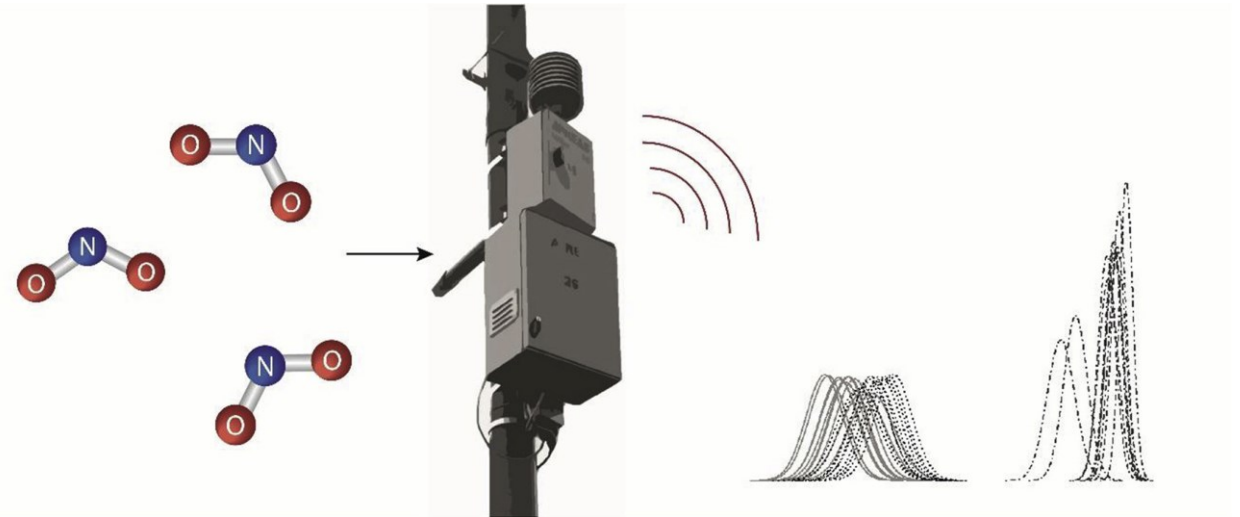
Normal Firms vs. Outlier Firms



9.2 이상치 탐지 및 처리 실습

Sample Dataset: Air Quality

- 실습 데이터: **Air Quality Dataset** ([link](#))
 - 이탈리아 도시에 설치된 다중 에어 센서 데이터
 - 공기 질 관련 11개의 변수로 구성(시간별 평균)
 - 일산화탄소(CO)
 - 산화질소(NO_x)
 - 비메탄계 탄화수소(NMHC)
 - 이산화질소(NO₂)
 - 상대습도(RH), 절대습도(AH) 등



Source: ScienceDirect

Datetime,CO(GT),PT08.S1(CO),PT08.S2(NMHC),NOx(GT),PT08.S3(NOx),NO2(GT),PT08.S4(NO2),PT08.S5(O3),RH,AH,C6H6(GT)

2004-03-10	18:00:00	2.6	1360.0	1046.0	166.0	1056.0	113.0	1692.0	1268.0	48.9	0.7578	11.9
2004-03-10	19:00:00	2.0	1292.0	955.0	103.0	1174.0	92.0	1559.0	972.0	47.7	0.7255	9.4
2004-03-10	20:00:00	2.2	1402.0	939.0	131.0	1140.0	114.0	1555.0	1074.0	54.0	0.7502	9.0
2004-03-10	21:00:00	2.2	1376.0	948.0	172.0	1092.0	122.0	1584.0	1203.0	60.0	0.7867	9.2
2004-03-10	22:00:00	1.6	1272.0	836.0	131.0	1205.0	116.0	1490.0	1110.0	59.6	0.7888	6.5
2004-03-10	23:00:00	1.2	1197.0	750.0	89.0	1337.0	96.0	1393.0	949.0	59.2	0.7848	4.7
2004-03-11	00:00:00	1.2	1185.0	690.0	62.0	1462.0	77.0	1333.0	733.0	56.8	0.7603	3.6
2004-03-11	01:00:00	1.0	1136.0	672.0	62.0	1453.0	76.0	1333.0	730.0	60.0	0.7702	3.3
2004-03-11	02:00:00	0.9	1094.0	609.0	45.0	1579.0	60.0	1276.0	620.0	59.7	0.7648	2.3
2004-03-11	03:00:00	0.6	1010.0	561.0		1705.0		1235.0	501.0	60.2	0.7517	1.7
2004-03-11	04:00:00		1011.0	527.0	21.0	1818.0	34.0	1197.0	445.0	60.5	0.7465	1.3
2004-03-11	05:00:00	0.7	1066.0	512.0	16.0	1918.0	28.0	1182.0	422.0	56.2	0.7366	1.1
2004-03-11	06:00:00	0.7	1052.0	553.0	34.0	1738.0	48.0	1221.0	472.0	58.1	0.7353	1.6
2004-03-11	07:00:00	1.1	1144.0	667.0	98.0	1490.0	82.0	1339.0	730.0	59.6	0.7417	3.2
2004-03-11	08:00:00	2.0	1333.0	900.0	174.0	1136.0	112.0	1517.0	1102.0	57.4	0.7408	8.0
2004-03-11	09:00:00	2.2	1351.0	960.0	129.0	1079.0	101.0	1583.0	1028.0	60.6	0.7691	9.5
2004-03-11	10:00:00	1.7	1233.0	827.0	112.0	1218.0	98.0	1446.0	860.0	58.4	0.7552	6.3
2004-03-11	11:00:00	1.5	1179.0	762.0	95.0	1328.0	92.0	1362.0	671.0	57.9	0.7352	5.0
2004-03-11	12:00:00	1.6	1236.0	774.0	104.0	1301.0	95.0	1401.0	664.0	66.8	0.7951	5.2
2004-03-11	13:00:00	1.9	1286.0	869.0	146.0	1162.0	112.0	1537.0	799.0	76.4	0.8393	7.3
2004-03-11	14:00:00	2.9	1371.0	1034.0	207.0	983.0	128.0	1730.0	1037.0	81.1	0.8736	11.5
2004-03-11	15:00:00	2.2	1310.0	933.0	184.0	1082.0	126.0	1647.0	946.0	79.8	0.8778	8.8
2004-03-11	16:00:00	2.2	1292.0	912.0	193.0	1103.0	131.0	1591.0	957.0	71.2	0.8569	8.3
2004-03-11	17:00:00	2.9	1383.0	1020.0	243.0	1008.0	135.0	1719.0	1104.0	67.6	0.8185	11.2
2004-03-11	18:00:00	4.8	1581.0	1319.0	281.0	799.0	151.0	2083.0	1409.0	64.2	0.8065	20.8
2004-03-11	19:00:00	6.9	1776.0	1488.0	383.0	702.0	172.0	2333.0	1704.0	69.3	0.8319	27.4
2004-03-11	20:00:00	6.1	1640.0	1404.0	351.0	743.0	165.0	2191.0	1654.0	67.8	0.8133	24.0
2004-03-11	21:00:00	3.9	1313.0	1076.0	240.0	957.0	136.0	1707.0	1285.0	64.0	0.7419	12.8

.csv 파일 내용

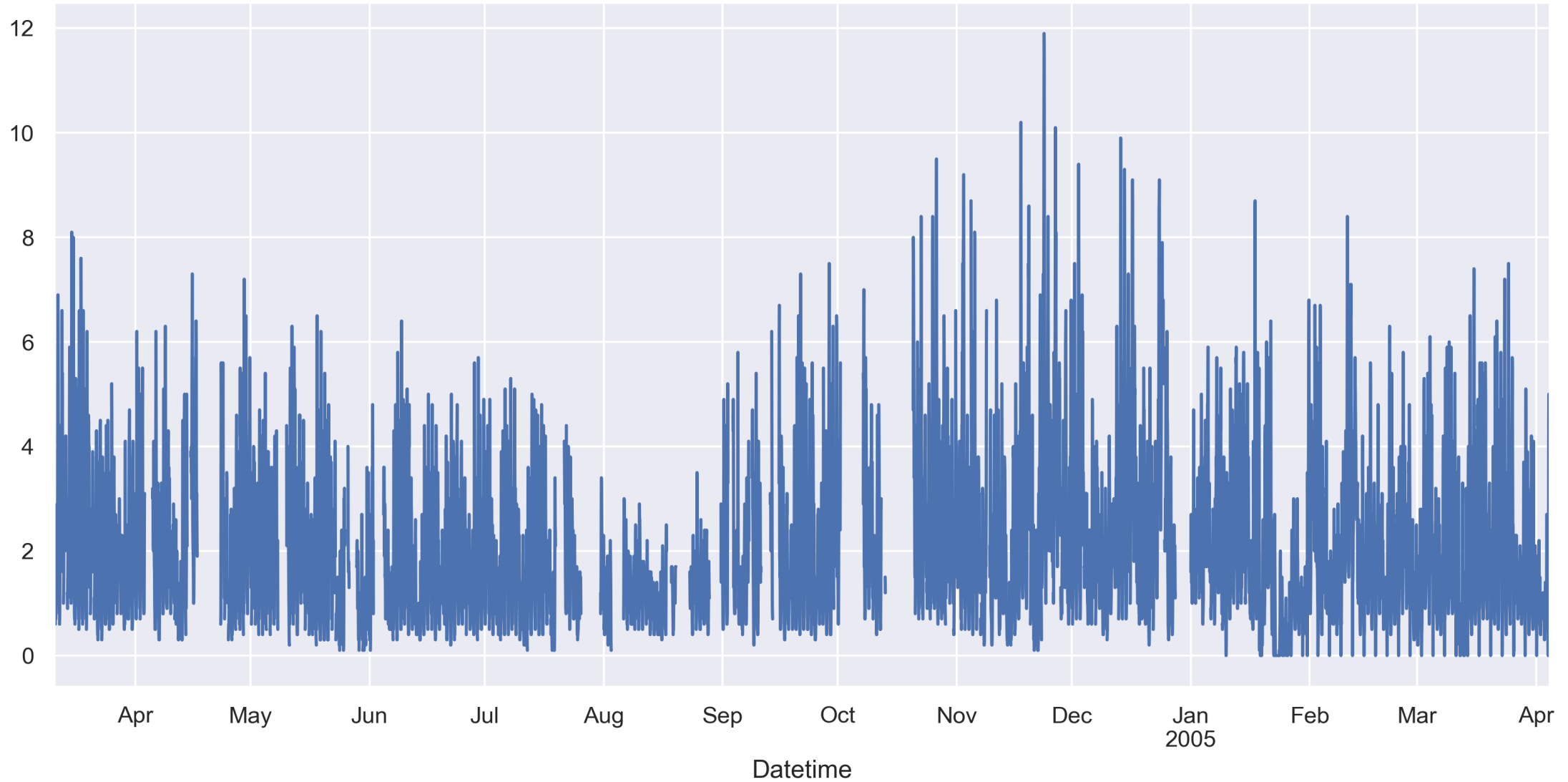
Load the Dataset



- 데이터셋 로딩

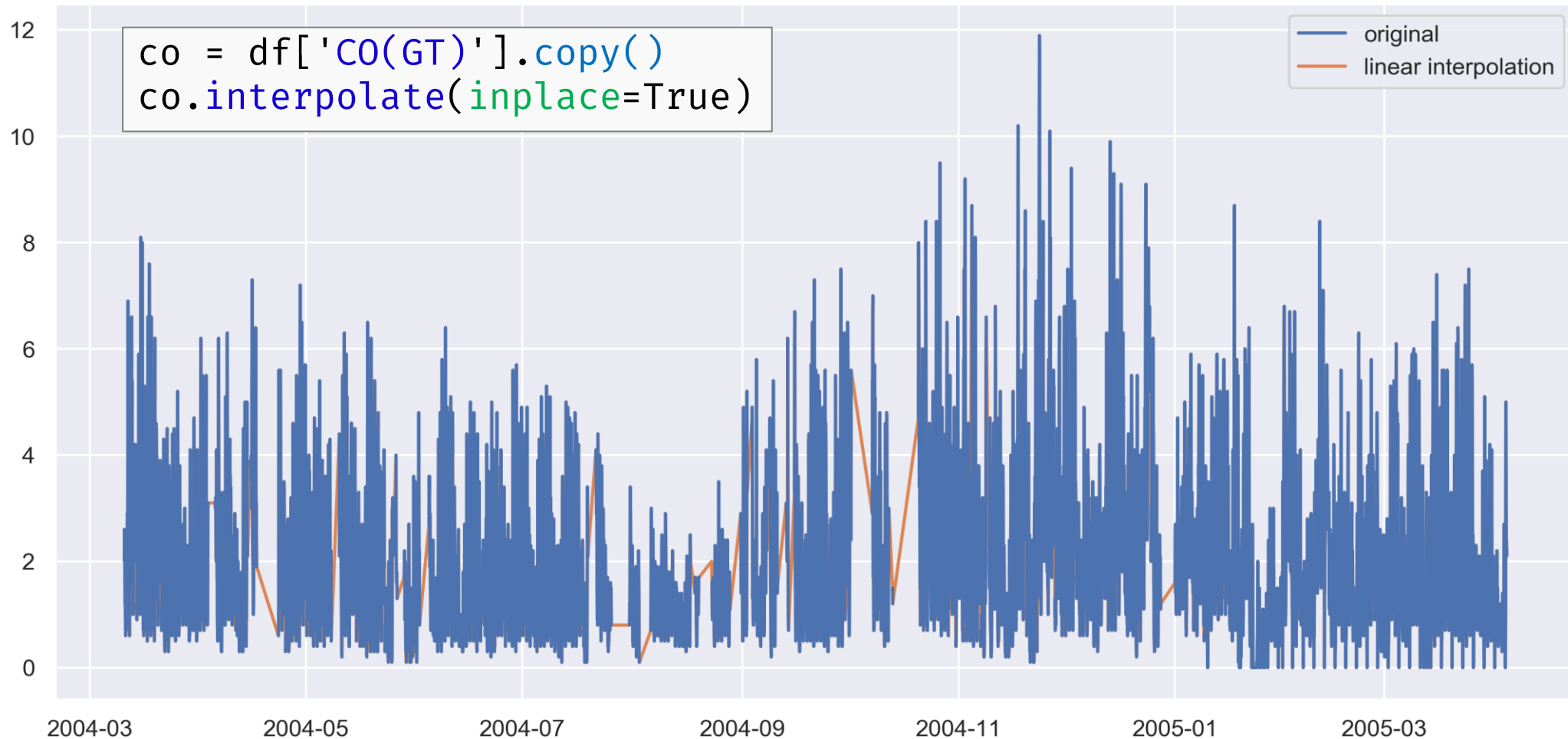
```
def parser(x):  
    return datetime.strptime(x, '%Y-%m-%d %H:%M:%S')  
  
path = './data/'  
file = 'AirQualityUCI_refined.csv'  
  
df = pd.read_csv(  
    path + file,  
    index_col=[0],  
    parse_dates=[0],  
    date_parser=parser  
)  
  
df.head()
```

```
df['CO(GT)'].plot()
```



Missing Data Reconstruction: Linear Interpolation

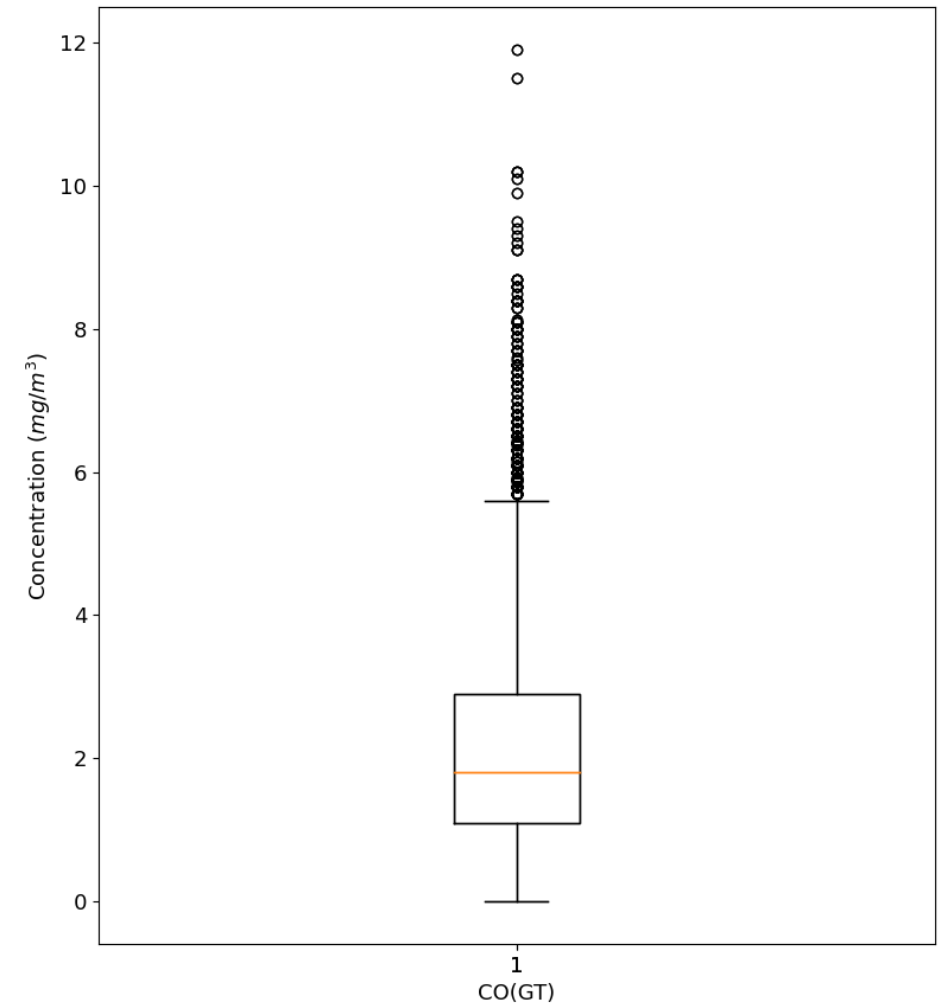
- 결측 데이터 처리: 선형보간



Visualize Outliers: Boxplot

- 이상치 시각화: 박스플롯
 - Q1, Q2, Q3
 - upper fence(상한), lower fence(하한)
 - maximum, minimum

```
plt.boxplot(co)
plt.xlabel('CO(GT)')
plt.ylabel('Concentration (mg/m3)')
```



Visualize Outliers: Scatter Plot



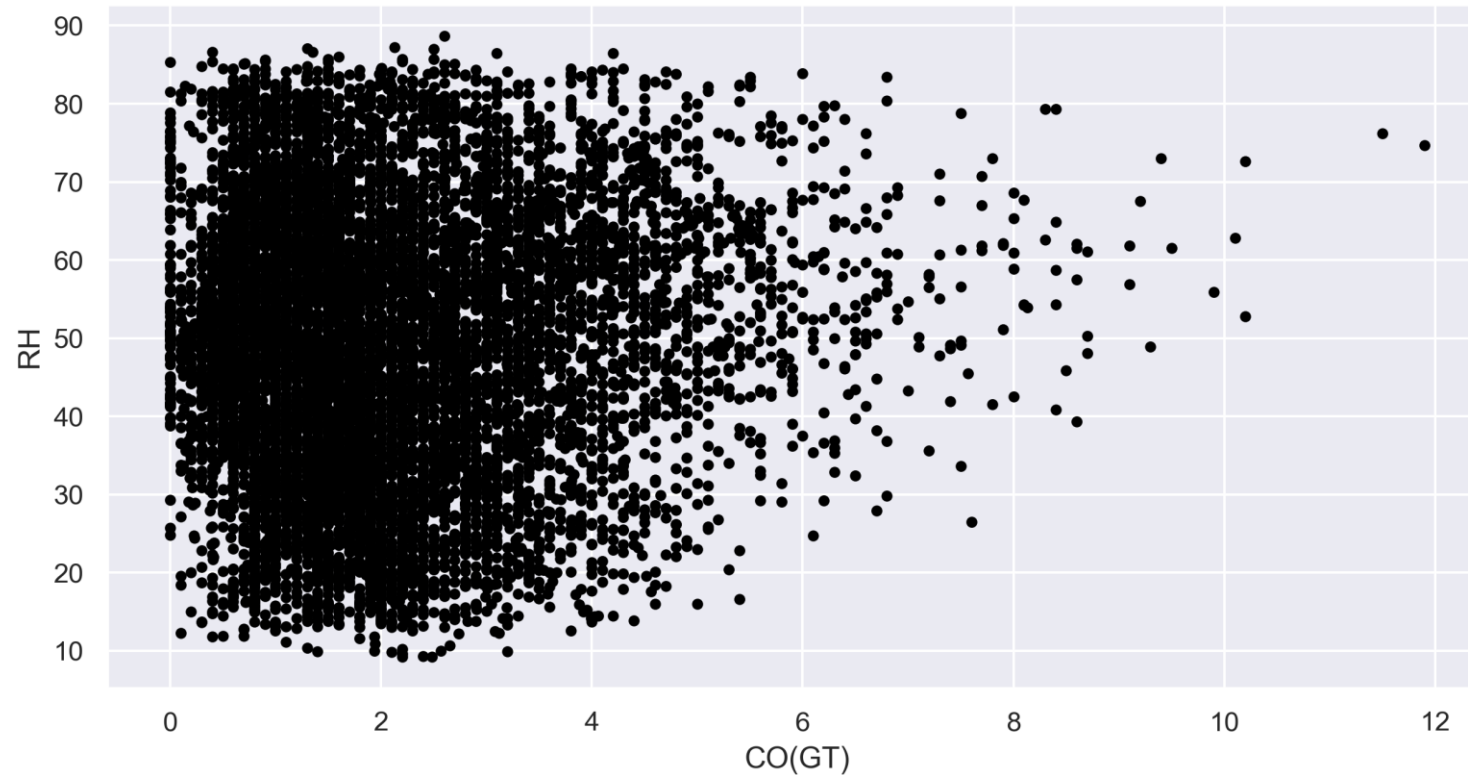
- 이상치 시각화: 산포도
 - 상관행렬을 통해 이산화탄소와 상관성이 높은/낮은 변수 선택

```
corr_matrix = df.corr()
```

	CO(GT)	PT08.S1(CO)	PT08.S2(NMHC)	NOx(GT)	PT08.S3(NOx)	NO2(GT)	PT08.S4(NO2)	PT08.S5(O3)	RH	AH	C6H6(GT)
CO(GT)	1.000000	0.877203	0.914973	0.792557	-0.701703	0.679262	0.639470	0.851403	0.040218	0.065809	0.845144
PT08.S1(CO)	0.877203	1.000000	0.892964	0.713654	-0.771938	0.641529	0.682881	0.899324	0.114606	0.135324	0.883795
PT08.S2(NMHC)	0.914973	0.892964	1.000000	0.704435	-0.796703	0.646245	0.777254	0.880578	-0.090380	0.186933	0.981950
NOx(GT)	0.792557	0.713654	0.704435	1.000000	-0.655707	0.763111	0.233731	0.787046	0.221032	-0.149323	0.626638
PT08.S3(NOx)	-0.701703	-0.771938	-0.796703	-0.655707	1.000000	-0.652083	-0.538468	-0.796569	-0.056740	-0.232017	-0.735744
NO2(GT)	0.679262	0.641529	0.646245	0.763111	-0.652083	1.000000	0.157360	0.708128	-0.091759	-0.335022	0.544039
PT08.S4(NO2)	0.639470	0.682881	0.777254	0.233731	-0.538468	0.157360	1.000000	0.591144	-0.032188	0.629641	0.765731
PT08.S5(O3)	0.851403	0.899324	0.880578	0.787046	-0.796569	0.708128	0.591144	1.000000	0.124956	0.070751	0.865689
RH	0.040218	0.114606	-0.090380	0.221032	-0.056740	-0.091759	-0.032188	0.124956	1.000000	0.167971	-0.061681
AH	0.065809	0.135324	0.186933	-0.149323	-0.232017	-0.335022	0.629641	0.070751	0.167971	1.000000	0.167972
C6H6(GT)	0.845144	0.883795	0.981950	0.626638	-0.735744	0.544039	0.765731	0.865689	-0.061681	0.167972	1.000000

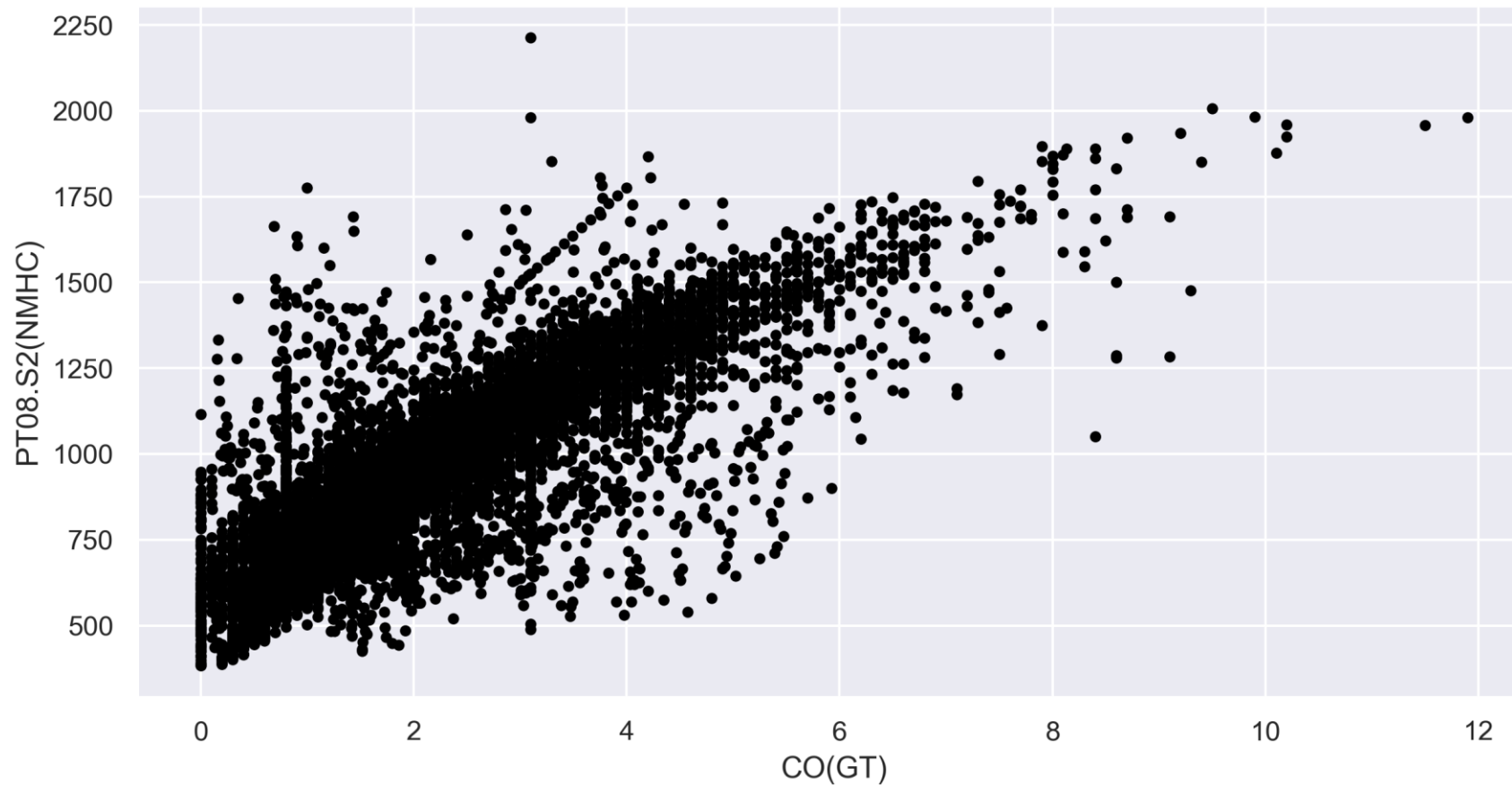
Visualize Outliers: Scatter Plot

- 상관성이 적은 두 변수: 이산화탄소(CO)와 상대습도(RH)
 - 두 변수간에 뚜렷한 패턴이 없어 시각적으로 이상치를 식별하기 어려움



Visualize Outliers: Scatter Plot

- 상관성이 높은 두 변수: 이산화탄소(CO)와 비메탄 탄화수소(NMHC)
 - 시각적으로 이상치를 식별하기 용이함



Outlier Detection: IQR-Based Method



- 사분위수 범위(IQR) 기반 이상치 탐지

- Q1, Q2, Q3 계산

```
q1 = co.quantile(0.25)
median = co.quantile(0.5)
q3 = co.quantile(0.75)
print(q1, median, q3)
```

```
1.1 1.8 2.9
```

- IQR, 상한, 하한 계산

```
iqr = q3-q1
upper_fence = q3 + 1.5*iqr
lower_fence = q1 - 1.5*iqr
print(upper_fence, lower_fence)
```

```
5.6 -1.5999999999999996
```


Outlier Detection: IQR-Based Method



- 이상치 탐지: 상한 보다 크거나 하한 보다 작은 값
 - `loc` 인덱서를 이용한 필터링
 - 음수가 존재할 수 없음(lower fence = 0)

```
outliers = co.loc[(co > upper_fence) | (co < 0)]  
outliers
```

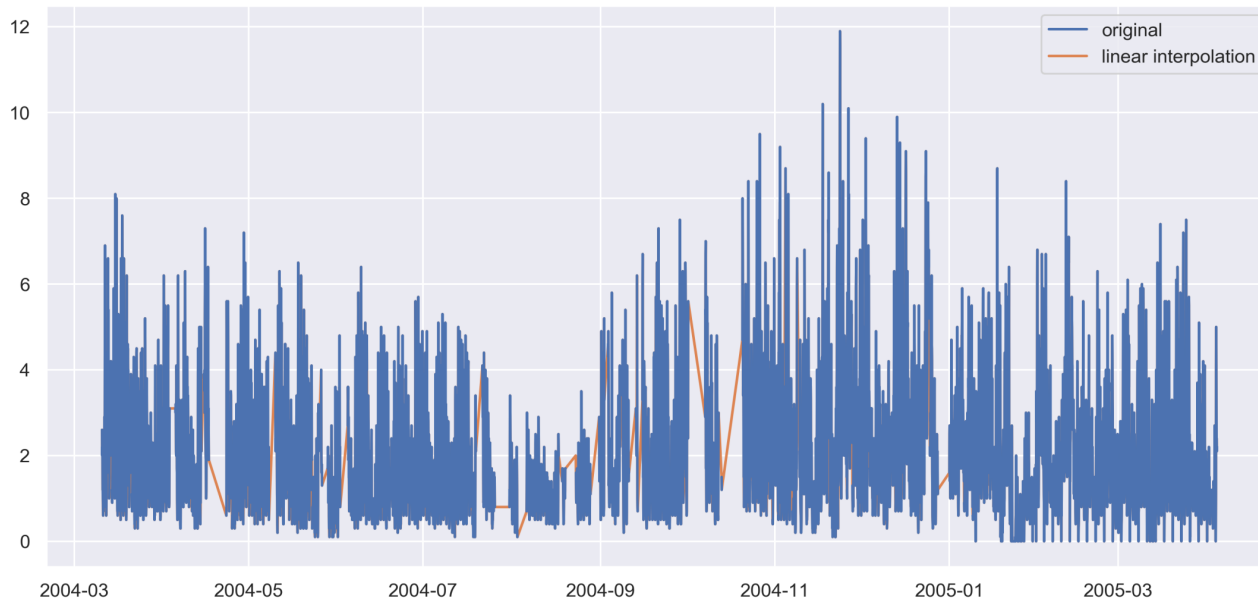
```
Datetime  
2004-03-11 19:00:00    6.9  
2004-03-11 20:00:00    6.1  
2004-03-12 20:00:00    6.6  
2004-03-14 20:00:00    5.9  
2004-03-15 09:00:00    8.1
```

⋮

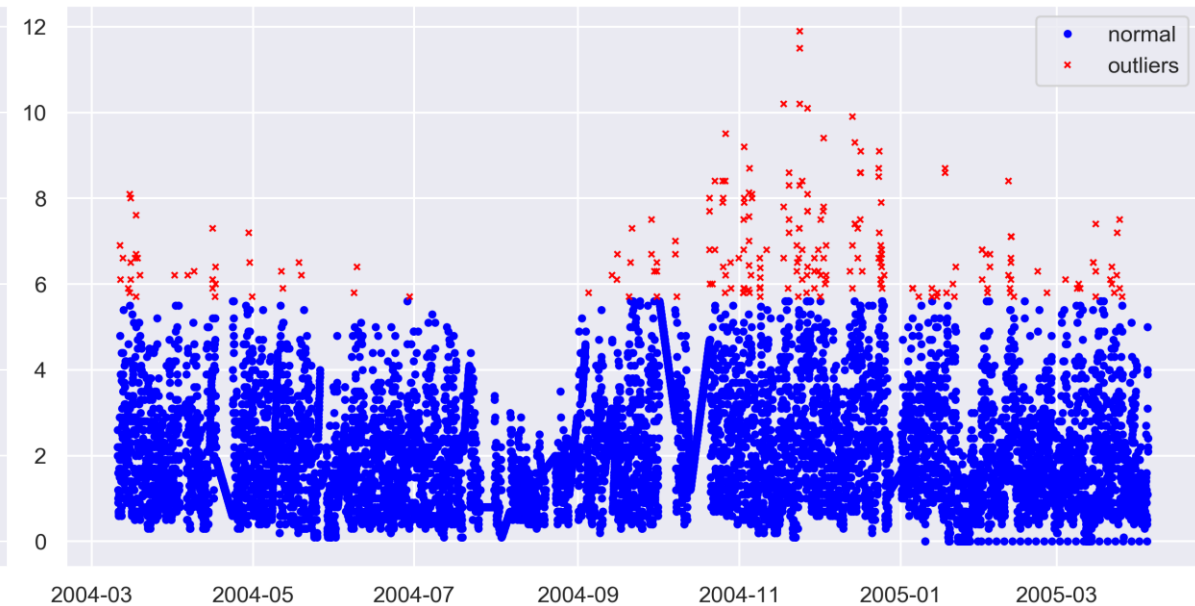
Outlier Detection: IQR-Based Method



- 이상치 탐지 결과



원본 데이터

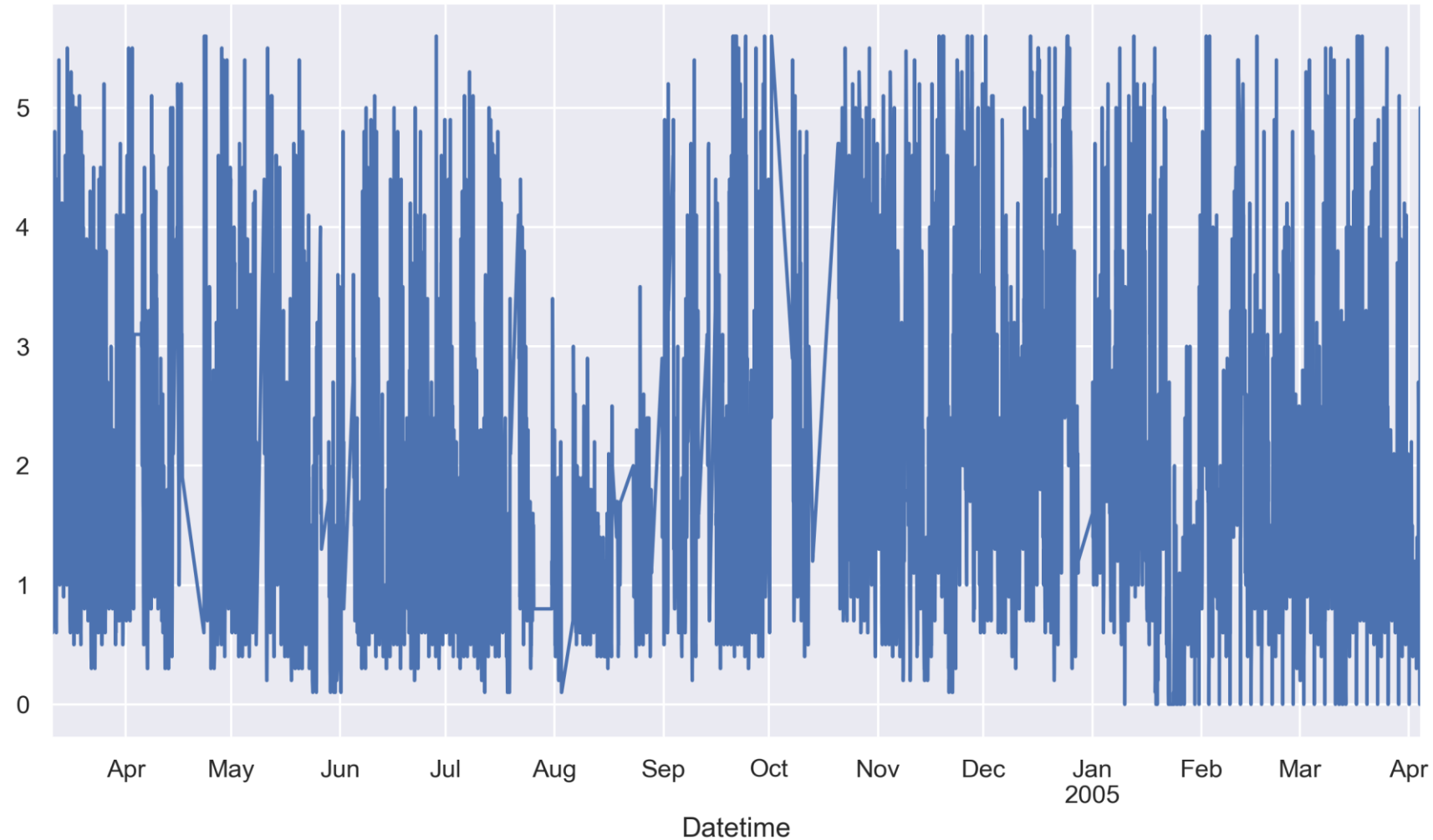


정상 데이터와 이상치

Outlier Detection: IQR-Based Method

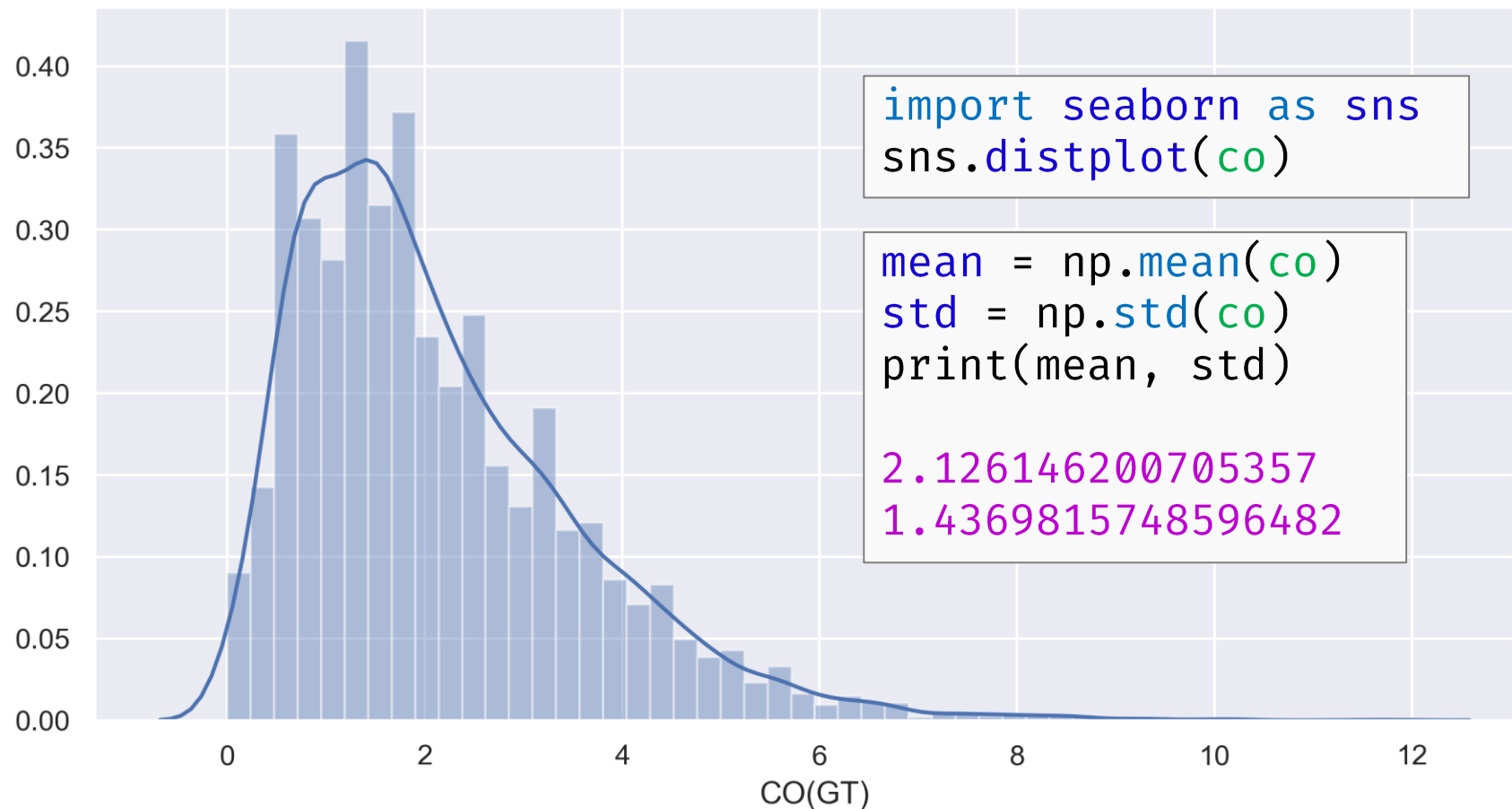


- 후처리: 이상치 제거 후 보간



Outlier Detection: Z-Scores Based Method

- **Z-Score** 기반 이상치 탐지
 - Z-Score 방법은 정규 분포를 가정(모수적 방법)



Outlier Detection: Z-Scores Based Method



- **Z-Score** 기반 이상치 탐지
 - 임계치(thres) 지정 후 각 데이터에 대한 이상치 여부 판단

```
outliers = []
thres = 3    # Z-score threshold

for i in co:
    z_score = (i-mean) / std
    if (np.abs(z_score) > thres):
        print(z_score)
        outliers.append(i)
```

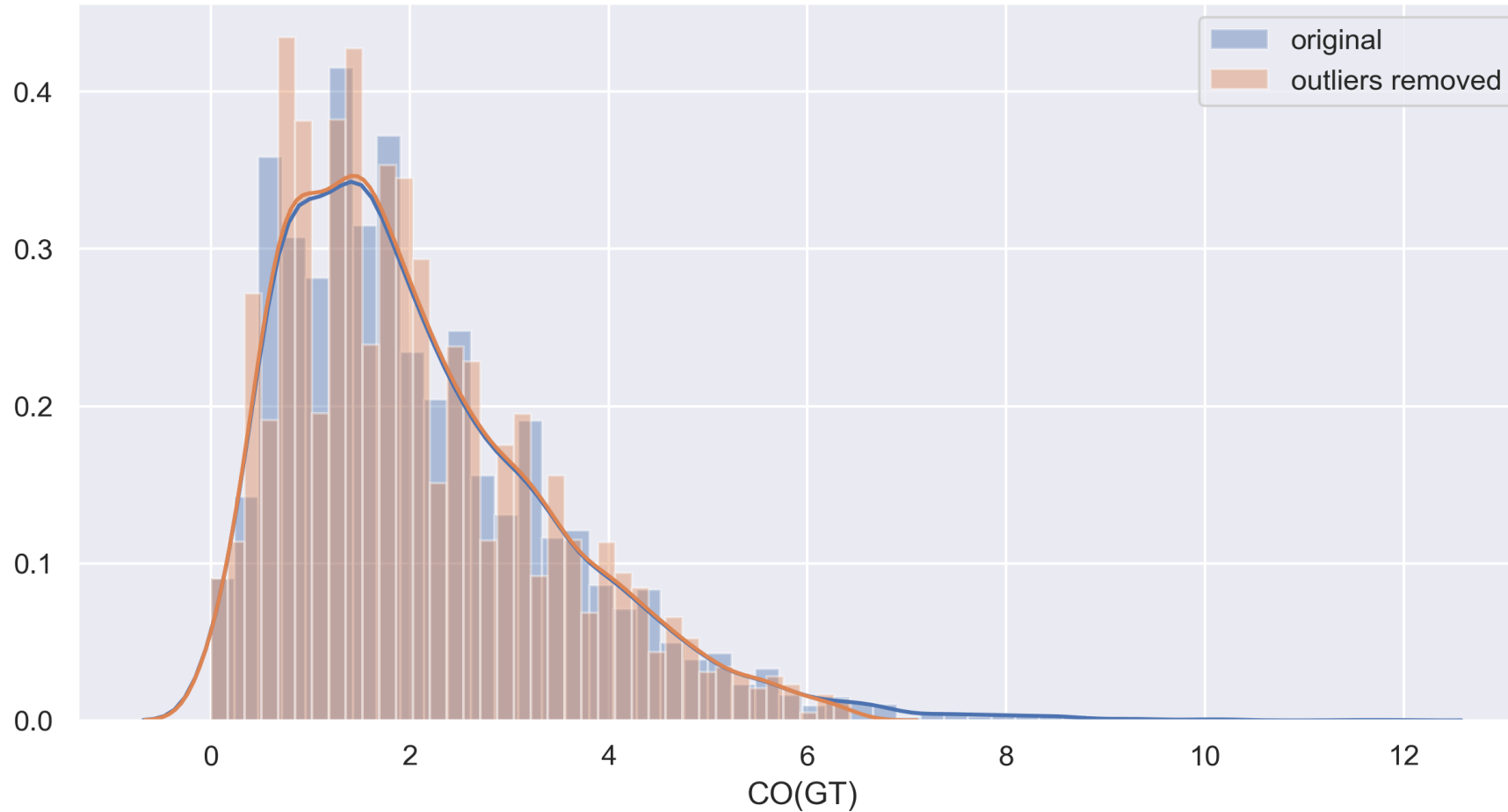
```
3.3221398818289734
3.113368937755246
4.157223658123882
4.087633343432639
```

⋮

Outlier Detection: Z-Scores Based Method



- 이상치 제거 후 분포 비교

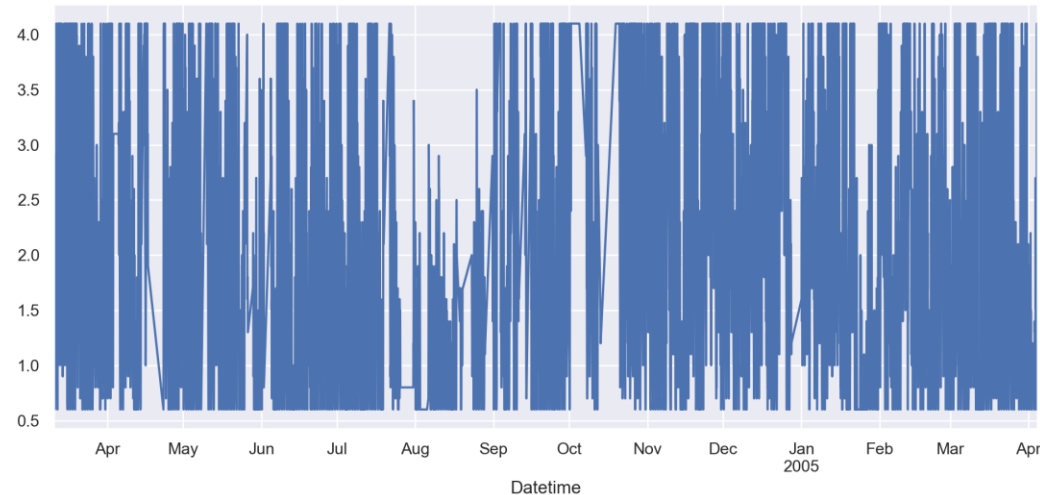
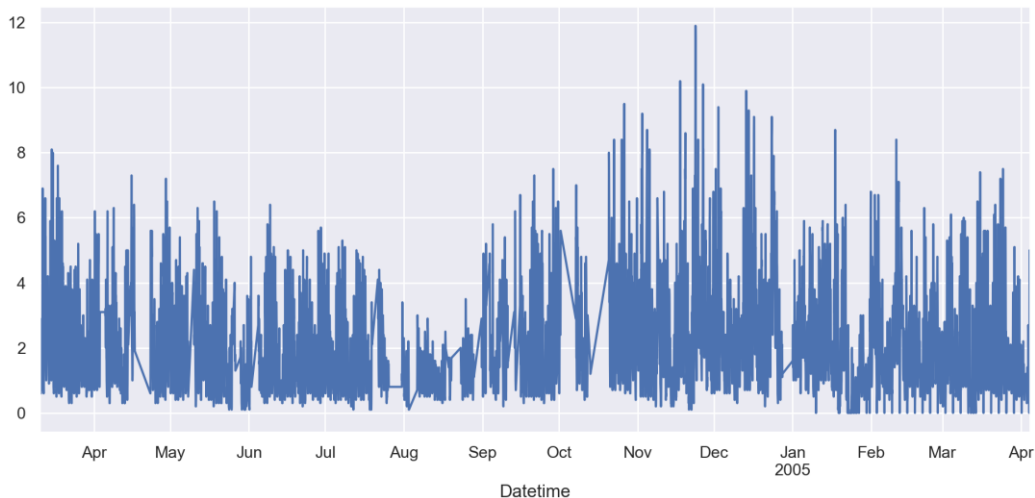


thres 를 조정해서
다른 결과도 시각화 해보기

Flooring and Capping

- 그 외의 이상치 처리 방법: **분위수(quantile)** 기반의 값 대체
 - **Flooring**: 작은 값을 가지는 이상치들을 낮은 분위수(예: 하위 10%)의 값으로 대체
 - **Capping**: 큰 값을 가지는 이상치들을 높은 분위수(예: 상위 10%)의 값으로 대체

```
floor = co.quantile(0.1)
cap = co.quantile(0.9)
co.loc[co < floor] = floor
co.loc[co > cap] = cap
```



References



- 아이.엠.김비서, [2과목] 빅데이터 탐색 – 데이터의 정제, 결측값, 이상값, 티스토리 ([link](#))
- Jen, [빅분기] 빅데이터 분석기사 필기시험 벼락치기 / 3회차 대비공부, 네이버블로그 ([link](#))
- 조대협, 클러스터링 #3 – DBSCAN (밀도 기반 클러스터링), 티스토리 ([link](#))
- Air Quality Data Set, UCI Machine Learning Repository ([link](#))
- Brownlee, J., 4 Automatic Outlier Detection Algorithms in Python, Machine Learning Mastery ([link](#))
- Santoyo, S., A Brief Overview of Outlier Detection Techniques, towards data science ([link](#))
- Comparing anomaly detection algorithms for outlier detection on toy datasets, scikit-learn ([link](#))
- TheDataPost, DBSCAN Explanation and Visualization, YouTube ([link](#))
- Bonthu, H., Detecting and Treating Outliers | Treating the odd one out!, Analytics Vidhya ([link](#))
- Goyal, C., Feature Engineering – How to Detect and Remove Outliers (with Python Code), Analytics Vidhya ([link](#))
- 안현, Isolation Forest, DEL@Hanshin ([link](#))
- 도도댕, Isolation Forest (for anomaly detection), 티스토리 ([link](#))
- Y. Yoon, Isolation Forest Anomaly Detection – Identify Outliers, Medium ([link](#))



수고하셨습니다.